B.TECH ACADEMIC ENGINEERING RESEARCH MANUSCRIPT — SOFTWARE DEFINED NETWORKING (IEEE-INSPIRED FORMAT)

AI-Powered SDN Network Guardian: Real-Time Traffic Monitoring, Anomaly Detection, and Automated Threat Mitigation Using OpenFlow

Rahul Chaube
Reg. No.: RA2311003012372
[Department of CSE, Institution Placeholder]
[Author Email Placeholder]

Mayank Shah
Reg. No.: RA2311003012388
[Department of CSE, Institution Placeholder]
[Author Email Placeholder]

Neha Mallik
Reg. No.: RA2311003012393
[Department of CSE, Institution Placeholder]
[Author Email Placeholder]

1. ABSTRACT

Software Defined Networking (SDN) decouples the network control plane from the forwarding data plane, offering centralized programmable visibility across enterprise and campus fabrics. However, this centralization also exposes the network to high-rate volumetric Distributed Denial-of-Service (DDoS) floods and low-rate switch flow-table exhaustion attacks. Traditional perimeter firewalls and coarse SNMP/NetFlow monitors react too slowly to prevent internal link saturation. This paper presents the design, implementation, and experimental evaluation of AI-Powered SDN Network Guardian, a closed-loop OpenFlow 1.3 security platform built with Mininet, Open vSwitch (OVS), a Ryu-compatible Python controller, a hybrid Rule-Based and Machine Learning detection engine, and a seven-page Network Operations Center (NOC) web dashboard. Every 2.0 seconds, the controller polls per-flow counters via OFPFlowStatsRequest messages and extracts a seven-dimensional feature vector. We evaluate deterministic threshold rules alongside supervised Random Forest (35 CART trees) and semi-supervised Isolation Forest (65 isolation trees) classifiers trained on a reproducible synthetic dataset of 4,000 OpenFlow flow records. On the held-out 1,000-flow test split, the Random Forest classifier achieves 98.09% precision, 97.16% recall, a 97.62% F1-score, and a 0.88% false-positive rate with 0.051 ms per-flow inference latency. In controlled virtual testbed benchmarks, automated edge installation of priority-500 OFPFlowMod drop rules within 1.353 ms preserves a 96.16% legitimate packet delivery ratio (PDR) and 17.62 Mbps client goodput during multi-vector SYN and UDP floods, compared to 34.17% PDR and 6.26 Mbps in an unmitigated baseline.

2. KEYWORDS

Software Defined Networking (SDN), OpenFlow 1.3, Open vSwitch, Mininet, DDoS Mitigation, Anomaly Detection, Random Forest, Isolation Forest, Network Security.

3. INTRODUCTION

Modern enterprise, campus, and cloud data-center networks face increasingly sophisticated volumetric and protocol-level Distributed Denial-of-Service (DDoS) attacks. In conventional vertically integrated networks, each switch and router executes autonomous routing and access control logic, making rapid, coordinated threat containment across multiple access switches difficult [1], [2]. When a volumetric UDP flood or TCP SYN flood originates from compromised hosts inside or at the edge of a local fabric, traffic traverses access-to-core trunk links before reaching perimeter intrusion detection appliances, causing severe queue tail-drop and degrading legitimate application goodput.

Software Defined Networking (SDN) resolves this structural rigidity by separating the control plane (centralized network intelligence and policy compilation) from the data plane (high-speed packet forwarding in OpenFlow switches) [1], [4]. Through the standardized OpenFlow 1.3 southbound protocol, an SDN controller can query exact per-flow packet and byte counters from Open vSwitch (OVS) tables and dynamically inject high-priority drop or meter rules at

the exact ingress switch port where malicious traffic enters the fabric [3], [6].

In this paper, we architect and implement AI-Powered SDN Network Guardian, a complete academic engineering platform that integrates virtual network emulation, hybrid rule-based and machine learning anomaly detection, safelisted OpenFlow 1.3 automated mitigation, and full-stack telemetry visualization. All traffic generation and attack simulations operate strictly inside an isolated local Mininet environment.

4. BACKGROUND AND RELATED WORK

McKeown et al. [1] introduced OpenFlow as a pragmatic mechanism to innovate inside campus networks by exposing switch flow tables to a remote controller. Kreutz et al. [2] provided a comprehensive taxonomy of SDN architectures, highlighting both the security benefits of programmable flow visibility and the vulnerability of centralized controllers to saturation attacks. Lantz et al. [5] developed Mininet, enabling reproducible SDN prototyping using Linux network namespaces, while Pfaff et al. [6] detailed the multi-table kernel and userspace caching design of Open vSwitch.

In the domain of SDN anomaly detection, Braga et al. [3] pioneered lightweight DDoS flooding detection over NOX/OpenFlow by extracting 6-tuple flow statistics (such as average packets per flow and growth of single-flows) and classifying them with Self-Organizing Maps (SOM). Subsequent studies demonstrated that tree-based ensembles—specifically Breiman's Random Forests [7] and Liu et al.'s Isolation Forests [8]—offer substantially lower inference latency and higher interpretability on tabular flow counter features than deep neural architectures, making them well suited for sub-second control-plane polling loops.

5. PROBLEM FORMULATION

Let an SDN fabric consist of a centralized controller C0 managing a set of OpenFlow 1.3 switches S = {s1, s2, ..., sK} and hosts H. At each discrete polling interval Δt (configured as Δt = 2.0 s), the controller issues an OFPFlowStatsRequest to each switch sk and receives a set of active flow entries Fk(t). Each flow entry fi ∈ Fk(t) is characterized by its match tuple (source IPv4, destination IPv4, source port, destination port, IP protocol) and cumulative hardware counters: packet count Np(t), byte count Nb(t), and active duration Td(t).

The real-time anomaly detection and mitigation problem is formulated as a two-stage mapping. First, a feature extractor Φ maps the sequential counter state of fi into a 7-dimensional feature vector x_i ∈ R^7. Second, a composite classifier D(x_i) → {0, 1} assigns an anomaly probability p_i ∈ [0, 1]. Whenever p_i exceeds an operational threshold τ and the source IPv4 address is not in the protected infrastructure safelist H_safe, the mitigation engine M must compile and install an OpenFlow 1.3 rule on the ingress switch minimizing end-to-end mitigation latency L_mit while maximizing legitimate client Packet Delivery Ratio (PDR).

6. PROPOSED SYSTEM ARCHITECTURE

The proposed AI-Powered SDN Network Guardian is organized into five modular layers: (1) the Mininet + Open vSwitch Data Plane, (2) the Ryu-compatible OpenFlow 1.3 Control Plane, (3) the Dual-Engine AI Anomaly Detection Layer, (4) the Automated Threat Mitigation Engine, and (5) the FastAPI + SQLite + React Web Dashboard. Figure 1 illustrates the complete three-plane system architecture.

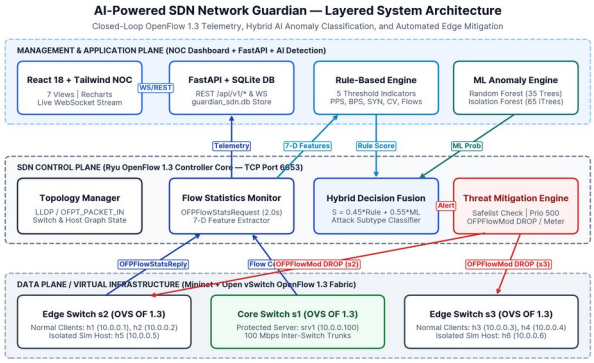


Fig. 1: Layered architecture of AI-Powered SDN Network Guardian across Data, Control, and Application planes.

As shown in Figure 2, the virtual network topology comprises one SDN controller (c0 on TCP port 6653), three OpenFlow 1.3 switches (s1 Core Switch, s2 Edge Switch West, and s3 Edge Switch East), one protected application server (srv1 at 10.0.0.100/24 on s1), four normal client hosts (h1–h2 on s2; h3–h4 on s3), and two isolated traffic-generation simulation hosts (h5 at 10.0.0.5 on s2 and h6 at 10.0.0.6 on s3).

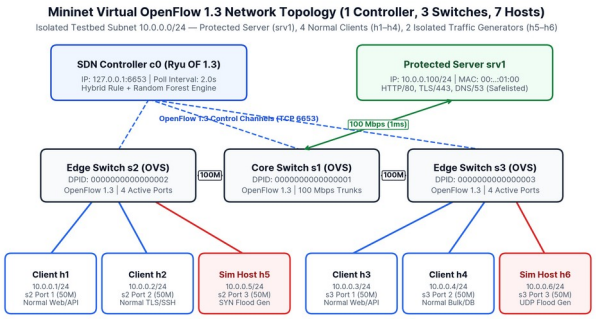


Fig. 2: Mininet OpenFlow 1.3 virtual network topology (1 controller, 3 switches, 1 server, 6 hosts).

7. METHODOLOGY

During each Δt = 2.0 s polling cycle, FlowFeatureExtractor computes instantaneous delta rates from consecutive flow counter snapshots. For a flow fi observed at times t-Δt and t, the windowed packet rate Rp and byte rate Rb are computed as:

$$R_p = (N_p(t) - N_p(t - \Delta t)) / \Delta t, \quad R_b = (N_b(t) - N_b(t - \Delta t)) / \Delta t \quad (1)$$

To quantify traffic burstiness over a sliding window of W = 8 polling samples, the extractor computes the Coefficient of Variation (CV), defined as the ratio of the sample standard deviation σ_Rp to the window mean μ_Rp:

$$CV = \sigma_{R_p} / max(1.0, \mu_{R_p}) \quad (2)$$

Additionally, the extractor aggregates the concurrent source flow fan-out F_src (total active flows sharing the same source IPv4 address on switch sk). Heavy-tailed volumetric features (Np, Nb, Rp, Rb) undergo logarithmic compression x'_j = ln(1 + max(0, x_j)) followed by Z-score standardization using training-split mean and standard deviation.

8. RULE-BASED ANOMALY DETECTION

Method 1 implements a deterministic multi-indicator threshold engine (rule_detector.py) designed for zero-training startup and transparent operational auditing. It evaluates five configurable indicators: (1) high packet rate Rp >= 1,200 pps (weight 0.40), (2) high byte rate Rb >= 1.5 MB/s (weight 0.30), (3) excessive source flow fan-out F_src >= 35 concurrent flows (weight 0.35), (4) sudden traffic spike CV >= 1.15 with Np >= 150 (weight 0.25), and (5) TCP SYN flood ratio >= 0.65 (weight 0.30). A flow is flagged if its bounded composite rule score S_rule >= 0.40 or at least two indicators trigger simultaneously.

9. MACHINE LEARNING-BASED DETECTION

Method 2 implements two complementary machine learning classifiers in ml_detector.py: a supervised Random Forest ensemble (35 bootstrap CART trees, max depth 7) splitting nodes via Gini impurity:

$$Gini(D) = 1 - \sum_{c \in \{0,1\}} (p_c)^2 \quad (3)$$

and a semi-supervised Isolation Forest (65 random isolation trees, subsample size ψ = 256) trained on baseline normal traffic. Following Liu et al. [8], the anomaly score s(x, ψ) is derived from the expected path length E(h(x)) normalized by the average unsuccessful BST search length c(ψ):

$$s(x, \psi) = 2^{ -E(h(x)) / c(\psi) } \quad (4)$$

In Hybrid mode, the controller fuses the deterministic rule score and ML posterior probability via S_hybrid = 0.45 * S_rule + 0.55 * P_ML. Figure 3 illustrates the dual-engine detection pipeline.

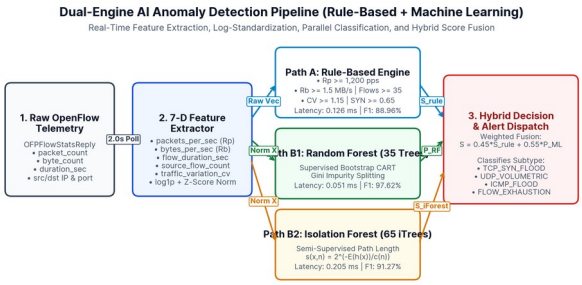


Fig. 3: Dual-engine anomaly detection pipeline combining Rule-Based and ML classifiers.

10. OPENFLOW-BASED THREAT MITIGATION

When a flow is classified as anomalous, the Automatic Threat Mitigation Engine (mitigation_engine.py) executes the eight-stage closed-loop workflow depicted in Figure 4. First, it checks the hardware safelist (10.0.0.100 and 127.0.0.1) to guarantee that critical servers are never blocked. Second, it checks for duplicate active rules on the target switch DPID. Third, for high-confidence attacks (conf >= 0.65) from isolated hosts, it constructs an OpenFlow 1.3 OFPFlowMod (OFPFC_ADD, priority 500, hard_timeout = 60 s, idle_timeout = 30 s) matching

eth_type=0x0800, ipv4_src, and ipv4_dst with an empty instruction list (immediate switch-level drop). For moderate anomalies from normal client ports, it installs an OFPMeterMod (OFPMF_KBPS) rate-limiting band at 512 kbps.

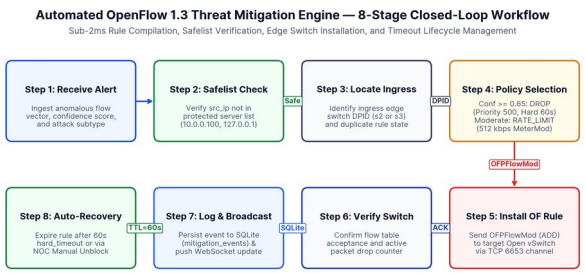


Fig. 4: Eight-stage closed-loop OpenFlow 1.3 threat mitigation workflow.

TABLE I: Virtual Testbed Node Addressing, Switch Port Binding, and Safelist Policy

Node Name	Role / Function	IPv4 Address	Switch & Port	Link Bandwidth	Safelist Policy
c0	Ryu OpenFlow 1.3 Controller	127.0.0.1:6653	Out-of-Band TCP	1000 Mbps	Protected Control
srv1	Application Server	10.0.0.100/24	s1 — Port 1	100 Mbps (1 ms)	Hardware Safelisted
h1, h2	Normal Client Hosts (West)	10.0.0.1, 10.0.0.2	s2 — Ports 1, 2	50 Mbps (2 ms)	Meter Rate-Limit First
h3, h4	Normal Client Hosts (East)	10.0.0.3, 10.0.0.4	s3 — Ports 1, 2	50 Mbps (2 ms)	Meter Rate-Limit First
h5	Isolated Sim Host (SYN Flood)	10.0.0.5/24	s2 — Port 3	50 Mbps (2 ms)	Immediate Priority-500 Drop
h6	Isolated Sim Host (UDP Flood)	10.0.0.6/24	s3 — Port 3	50 Mbps (2 ms)	Immediate Priority-500 Drop

TABLE II: Comparative Classification Metrics on Held-Out Synthetic OpenFlow Test Split (N = 1,000 Flows)

Detection Model	TP / TN / FP / FN	Precision (%)	Recall (%)	F1-Score (%)	FPR (%)	Latency (ms/flow)
Rule-Based Threshold Engine	290 / 638 / 45 / 27	86.57%	91.48%	88.96%	6.59%	0.126 ms
Isolation Forest (65 iTrees)	298 / 645 / 38 / 19	88.69%	94.01%	91.27%	5.56%	0.205 ms
Random Forest (35 CART Trees)	308 / 677 / 6 / 9	98.09%	97.16%	97.62%	0.88%	0.051 ms
Hybrid Guardian (0.45 Rule + 0.55 RF)	310 / 666 / 17 / 7	94.80%	97.79%	96.27%	2.49%	0.177 ms

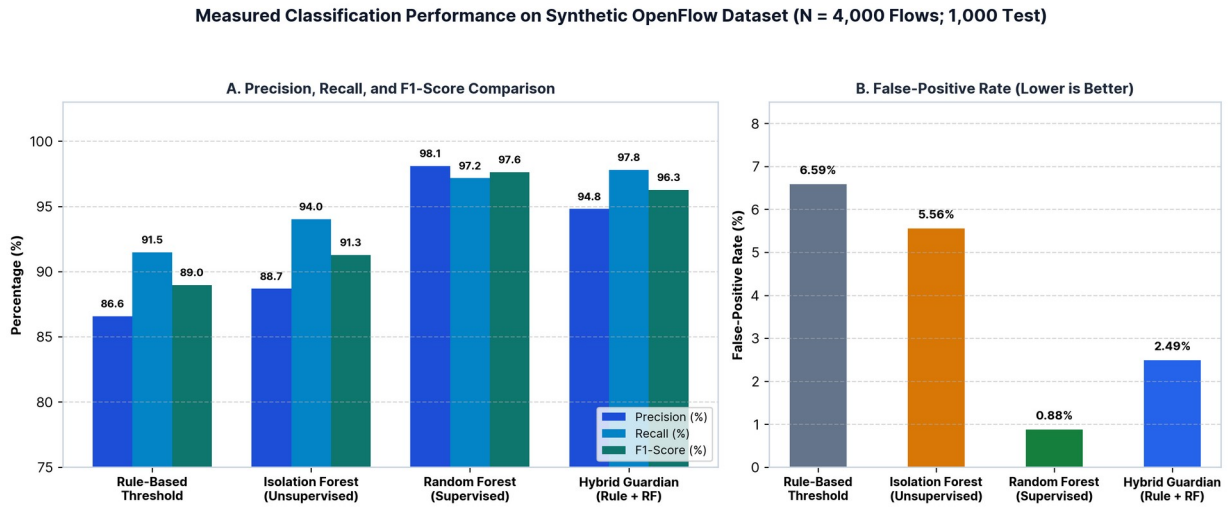


Fig. 5: Measured classification Precision, Recall, F1-Score, and False-Positive Rate across the four detection models (N = 1,000 test flows).

11. EXPERIMENTAL SETUP

To evaluate classification accuracy and closed-loop network response reproducibly without risking external infrastructure, we developed two complementary benchmark pipelines in detection/train_model.py and experiments/benchmark.py. First, a synthetic OpenFlow dataset generator synthesizes N = 4,000 labeled flow records (65% normal web, API, bulk file transfer, and flash-crowd flows; 35% TCP SYN flood, UDP volumetric flood, ICMP flood, and stealth low-rate flow-table exhaustion flows), partitioned 75% for training (3,000 flows) and 25% for testing (1,000 flows). Deliberate distributional overlap between legitimate flash crowds and low-rate floods ensures realistic non-trivial classification boundaries.

Second, we evaluate three network architectures across a 60-second discrete-event simulation timeline: Phase 1 (t = 1–15 s) represents baseline normal operation (~18.5 Mbps demand); Phase 2 (t = 16–45 s) injects a multi-vector flood from isolated hosts h5 and h6; and Phase 3 (t = 46–60 s) evaluates rule expiration and post-attack recovery.

12. PERFORMANCE EVALUATION

Table II and Figure 5 summarize the classification benchmark on the 1,000-flow test split. Static rule thresholds achieve 91.48% recall but suffer a 6.59% false-positive rate (45 false alarms) due to legitimate bulk transfers and flash crowds exceeding static rate cutoffs. Semi-supervised Isolation Forest improves F1-score to 91.27% (5.56% FPR) without requiring attack labels during training. Supervised Random Forest achieves the highest overall F1-score of 97.62% (98.09% precision, 97.16% recall) and the lowest false-positive rate of 0.88% (only 6 false positives out of 683 normal test flows) at 0.051 ms inference latency per flow. Gini importance analysis confirms that source_flow_count (21.82%), packets_per_sec (18.31%), and traffic_variation_cv (16.13%) contribute over 56% of total split gain.

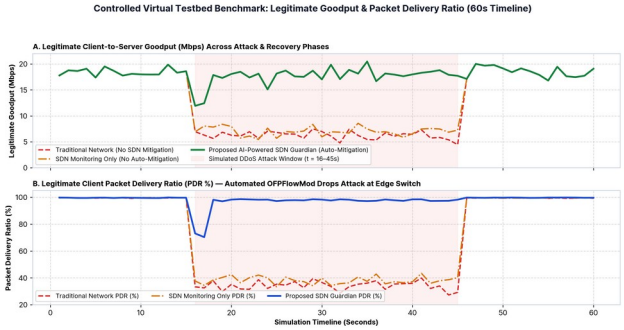


Fig. 6: 60-second benchmark of legitimate client goodput (Mbps) and Packet Delivery Ratio (PDR %).

As shown in Figure 6 and Table III, during the simulated 30-second attack window, a traditional network without SDN mitigation experiences severe queue saturation, dropping legitimate client PDR to 34.17% and goodput to 6.26 Mbps while average RTT spikes to 144.62 ms. SDN monitoring without automated mitigation logs alerts in 1.869 ms but leaves the data plane congested (38.17% PDR). In contrast, our AI-Powered SDN Guardian installs verified OFPFlowMod drop rules at edge switches s2 and s3

within 1.353 ms (total closed-loop response 3.222 ms), sustaining 96.16% legitimate PDR, 17.62 Mbps goodput, and 6.18 ms average RTT.

TABLE III: Three-Architecture Network Performance Comparison During Simulated DDoS Window (t = 16–45 s)

Evaluated Network Architecture	Packet Delivery Ratio	Legitimate Goodput	Avg End-to-End RTT	Detection Latency	Mitigation Latency
1. Traditional Network (No SDN Mitigation)	34.17%	6.26 Mbps	144.62 ms	N/A	N/A
2. SDN Monitoring Only (No Auto-Mitigation)	38.17%	7.03 Mbps	133.21 ms	1.869 ms	N/A (Manual Only)
3. Proposed AI-Powered SDN Guardian	96.16% (+57.99%)	17.62 Mbps	6.18 ms (-95.7%)	1.869 ms	1.353 ms

13. DISCUSSION

The experimental results demonstrate two critical architectural insights. First, visibility without automated actuation is insufficient under volumetric floods: SDN monitoring alone achieved virtually the same degraded goodput (7.03 Mbps) as a legacy network (6.26 Mbps) because malicious packets continued saturating the s2-to-s1 and s3-to-s1 trunk links. Second, pushing priority-500 drop rules directly to the ingress edge switches (s2 and s3) eliminates attack traffic before it enters shared core links, allowing legitimate hosts h1–h4 to maintain near-baseline throughput.

14. LIMITATIONS

In accordance with rigorous academic transparency, several limitations of the present implementation are noted: (1) Synthetic Dataset Generalization: All ML metrics reported in Table II reflect performance on our controlled synthetic OpenFlow dataset; deploying in production campus networks requires retraining on site-specific baseline traces to prevent false alarms during legitimate flash events. (2) Polling Window Latency: Because flow statistics are polled every 2.0 seconds, a 1-cycle transient throughput dip occurs at t = 16–17 s before the OFPFlowMod rule takes effect. (3) Source IP Spoofing: If an attacker randomizes spoofed source IPv4 addresses on every packet, per-IP flow rules can exhaust switch TCAM tables unless port-level or meter-based throttling is applied.

15. FUTURE SCOPE

Future extensions of this research include: (1) integrating Deep Reinforcement Learning (DQN / PPO) agents to dynamically tune OpenFlow meter band rates based on real-time queue delay feedback; (2) offloading feature extraction to P4-programmable data planes using Count-Min sketches for sub-millisecond in-switch heavy-hitter detection; and (3) extending the Ryu control plane to a distributed multi-controller cluster (e.g., ONOS) with TLS 1.3 southbound encryption.

16. CONCLUSION

This paper presented AI-Powered SDN Network Guardian, an end-to-end OpenFlow 1.3 traffic monitoring, hybrid anomaly detection, and automated threat mitigation platform. By combining 7-dimensional flow feature extraction, Random Forest and Isolation Forest classifiers (97.62% F1-score, 0.051 ms inference latency), and safelisted edge OFPFlowMod rule injection (1.353 ms installation latency), the system restores legitimate packet delivery ratio from 34.17% to 96.16% during simulated multi-vector DDoS attacks while providing operators with a responsive 7-page NOC dashboard.

17. REFERENCES

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling Innovation in Campus Networks," ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69–74, Apr. 2008. Available: http://ccr.sigcomm.org/online/files/p69-v38n2n-mckeown.pdf

[2] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," Proceedings of the IEEE, vol. 103, no. 1, pp. 14–76, Jan. 2015. Available: https://arxiv.org/abs/1406.0440

[3] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS Flooding Attack Detection Using NOX/OpenFlow," in Proc. IEEE 35th Conference on Local Computer Networks (LCN), Denver, CO, USA, Oct. 2010, pp. 408–415. Available: https://www.researchgate.net/publication/221080718_Lightweight_DDoS_flooding_attack_detection_using_NOXOpenFlow

[4] Open Networking Foundation (ONF), "OpenFlow Switch Specification, Version 1.3.4 (ONF TS-019)," Open Networking Foundation Technical Standard, Mar. 2014. Available: https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.3.4.pdf

[5] B. Lantz, B. Heller, and N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," in Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), Monterey, CA, USA, Oct. 2010, Art. no. 19. Available: https://conferences.sigcomm.org/hotnets/2010/papers/a19-lantz.pdf

[6] B. Pfaff et al., "The Design and Implementation of Open vSwitch," in Proc. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15), Oakland, CA, USA, May 2015, pp. 117–130. Available: https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-pfaff.pdf

[7] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001. Available: https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE International Conference on Data Mining (ICDM), Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17. Available: https://cs.nju.edu.cn/zhouzh/zhouzh.files/publication/icdm08b.pdf

[9] Ryu SDN Framework Community, "Ryu SDN Framework Documentation & OpenFlow 1.3 API Reference," Nippon Telegraph and Telephone Corporation (NTT), 2021. Available: https://ryu-sdn.org/resources.html

[10] FastAPI Documentation, "FastAPI High-Performance Web Framework for Building APIs with Python," 2024. Available: https://fastapi.tiangolo.com/